



ArqUX vs. la industria

Evaluación técnica comparativa de un framework de gobernanza de agentes IA basado en MCP.

Protocolo reproducible de evaluación en 5 dimensiones (Core Engineering, Ecosistema, Operacional, Adopción y Sostenibilidad) aplicado a ArqUX y contrastado contra CrewAI, LangGraph, Microsoft AutoGen, OpenAI Agents SDK y Guardrails AI. Versión inicial de un benchmark pensado para evolucionar.

SCORE GLOBAL ARQUX

1.69

/ 5.00 ponderado · Ranking #6 de 6

AUDIENCIA Mixta técnica + negocio

METODO Likert 1-5 con pesos por dimensión

FECHA Julio 2026

Indice

1. Resumen ejecutivo	6
Hallazgos clave	6
Tabla de scoring final	6
Tres virtudes y tres debilidades criticas	7
Recomendacion de adopcion	7
Parte I - Protocolo de evaluacion	8
Capitulos de esta parte	8
2. Introduccion y contexto	9
2.1 Por que evaluar ArqUX	9
2.2 Contexto: el problema de la gobernanza de agentes	9
2.3 Hipotesis del benchmark	9
2.4 Objetivos del estudio	10
2.5 Alcance y audiencia	10
3. Metodologia del benchmark	11
3.1 Enfoque: tecnico pragmatico	11
3.2 Cinco dimensiones de evaluacion	11
3.3 Escala Likert 1-5 con anclajes	11
3.4 Scoring ponderado	12
3.5 Criterios de reproducibilidad	12
3.6 Limitaciones declaradas	12
4. Dimensiones y metricas detalladas	13

4.1 Dimension 1: Core Engineering (peso 30%)	13
Subcriterios	13
4.2 Dimension 2: Ecosistema (peso 20%)	13
Subcriterios	13
4.3 Dimension 3: Operacional (peso 20%)	14
Subcriterios	14
4.4 Dimension 4: Adopcion (peso 15%)	14
Subcriterios	14
4.5 Dimension 5: Sostenibilidad (peso 15%)	15
Subcriterios	15

5. Procedimiento de reproduccion 16

5.1 Diagrama del procedimiento	16
5.2 Paso 1: Captura de fuentes	16
5.3 Paso 2: Metricas publicas	16
5.4 Paso 3: Analisis de codigo	17
5.5 Paso 4: Scoring Likert 1-5	17
5.6 Paso 5: Calculo de score ponderado	17
5.7 Manejo de incertidumbre	17
5.8 Criterios de actualizacion del protocolo	18

Parte II - Analisis tecnico de ArqUX 19

Capitulos de esta parte	19
-----------------------------------	----

6. Analisis tecnico de ArqUX 20

6.1 Identidad y proposito	20
6.2 Stack tecnologico	20

6.3 Arquitectura: Hexagon Contract 20

6.4 Layout del codigo y patrones 21

6.5 API surface: tres vias de acceso 21

6.6 Pipeline operativo 22

6.7 Identidades y roles 22

6.8 Memoria CORTEX y learning loop 22

7. Identidad, madurez y ecosistema de ArqUX **23**

7.1 Metricas de madurez del proyecto 23

7.2 Autor y posicionamiento 23

7.3 Ecosistema y dependencias 24

7.4 Gaps auto-reconocidos 24

7.5 Deriva documental 24

Parte III - Benchmark comparativo **25**

Capitulos de esta parte 25

8. Soluciones de referencia **26**

8.1 Criterios de seleccion 26

8.2 CrewAI 26

8.3 LangGraph (LangChain) 26

8.4 Microsoft AutoGen 26

8.5 OpenAI Agents SDK 26

8.6 Guardrails AI 27

8.7 Tabla comparativa resumida 27

9. Analisis comparativo por dimension **28**

9.1 Core Engineering (peso 30%)	28
9.2 Ecosistema (peso 20%)	28
9.3 Operacional (peso 20%)	29
9.4 Adopcion (peso 15%)	29
9.5 Sostenibilidad (peso 15%)	30
Parte IV - Hallazgos y recomendaciones	31
Capitulos de esta parte	31
10. Matriz de scoring final y visualizaciones	32
10.1 Score ponderado final	32
10.2 Radar comparativo	32
10.3 Heatmap de scoring	33
10.4 Ranking final	34
10.5 Timeline de madurez	34
10.6 Diagrama comparativo de arquitectura	35
11. Hallazgos: virtudes y debilidades	36
11.1 Virtudes de ArqUX	36
11.2 Debilidades criticas	36
11.3 Matriz FODA	37
12. Recomendaciones accionables	38
12.1 Para equipos evaluando adoptar ArqUX	38
12.2 Para el autor de ArqUX	38
12.3 Para el protocolo de benchmark v1.1	39
13. Roadmap de evolucion del protocolo	40

13.1 Plan de versiones	40
13.2 Dependencias y riesgos del roadmap	40
13.3 Criterios de exito del protocolo	40
14. Conclusiones	41
Anexo A - Glosario tecnico	42
Anexo B - Fuentes verificadas y ficha de reproducibilidad	44
B.1 Fuentes verificadas (captura: 2026-07-08)	44
B.2 Comandos para re-extraer datos	45
B.3 Plantilla de captura para futuras versiones	45
B.4 Nota final de reproducibilidad	46

Control de versiones del protocolo. Esta es la version 1.0.0 del benchmark ArqUX, definida como protocolo inicial reproducible. El plan de evolucion contempla v1.1 (metricas cuantitativas de performance reproducibles) y v1.2 (analisis estatico + encuestas DX normalizadas), con una revision semestral de pesos y dimensiones. Ver Capitulo 13 para el roadmap completo.

1. Resumen ejecutivo

ArqUX es un framework de gobernanza para agentes de IA basado en el protocolo MCP (Model Context Protocol), desarrollado en Python por un unico autor y publicado por primera vez en PyPI el 8 de julio de 2026. Su tesis diferenciadora consiste en reemplazar las conversaciones de chat temporales por un modelo operativo gobernado: identidades con roles, contratos de comportamiento, memoria persistente, handoffs explicitos, Blueprints y evidencia verificable. Esta evaluacion aplica un protocolo reproducible de cinco dimensiones (Core Engineering, Ecosistema, Operacional, Adopcion y Sostenibilidad) y contrasta ArqUX contra cinco soluciones líderes de la industria: CrewAI, LangGraph, Microsoft AutoGen, OpenAI Agents SDK y Guardrails AI.

El hallazgo central es doble. Por un lado, ArqUX aporta una propuesta conceptualmente original: ninguna de las cinco soluciones de referencia combina de manera tan explicita identidad + contrato + evidencia sobre MCP. Por otro lado, su madurez empirica es irrelevante hoy: cero estrellas en GitHub, cero forks, cero issues, un unico contribuidor, dependencia de otro proyecto beta del mismo autor (codec-cortex), deriva documental auto-reconocida y un pipeline de CI insuficiente. El score ponderado final de ArqUX es 1.69/5.00, ubicandolo ultimo en el ranking detras de LangGraph (4.32), CrewAI (4.11), AutoGen (4.00), OpenAI Agents SDK (3.75) y Guardrails AI (3.73).

Hallazgos clave

SCORE ARQUX	RANKING	LIDER	BRECHA
1.69	#6	LangGrapp	2.63 pts
/ 5.00 ponderado	de 6 soluciones	h 4.32 / 5.00	vs lider del grupo

Tabla de scoring final

Solucion	Core (30 %)	Eco (20%)	Oper (20 %)	Adop (15 %)	Sost (15%)	Score final
ArqUX	3.2	1.0	1.5	0.5	1.0	1.69
CrewAI	3.8	4.5	4.0	4.5	4.0	4.11
LangGraph	4.5	4.5	4.2	4.0	4.2	4.32
Microsoft AutoGen	4.0	4.0	3.8	3.8	4.5	4.00
OpenAI Agents SDK	3.5	4.0	3.5	4.2	3.8	3.75
Guardrails AI	3.8	3.5	4.0	3.5	3.8	3.73

Tabla 1.1 - Scoring por dimension y score ponderado final (escala Likert 1-5).

Tres virtudes y tres debilidades criticas

Virtudes:

- **Tesis diferenciadora.** Unica solucion nativa-MCP que combina identidad, contrato, evidencia y memoria institucional en una sola capa.
- **Codigo razonablemente estructurado.** Registry pattern (HandlerSpec), permisos por rol (governor / executor / auditor), tests (~69-100) y dogfooding del propio repo.
- **Licencia permisiva y stack moderno.** Apache-2.0, Python 3.10+, dependencias maduras (pydantic, click, rich, mcp>=1.0.0).

Debilidades criticas:

- **Adopcion y comunidad nulas.** 0 estrellas, 0 forks, 0 issues, sin Discord/Discussions/sitio web/menciones externas. Bus factor = 1.
- **Deriva documental y gaps HIGH auto-reconocidos.** Inconsistencia entre 24 / 54 / 62 handlers; GAP-001 (dualidad brain.cortex) y GAP-002 (14 handlers sin documentar) marcados como HIGH por el propio autor.
- **Dependencia fragil y CI insuficiente.** Depende de codec-cortex (otro proyecto beta del mismo autor); los tests solo corren en pipeline de release, no en push/PR.

Recomendacion de adopcion

No adoptar ArqUX en produccion hoy. Su valor reside en la tesis conceptual (gobernanza nativa-MCP con identidad + contrato + evidencia), no en su madurez operativa. Equipos que evaluan gobernanza de agentes en produccion deben priorizar LangGraph o CrewAI. ArqUX merece vigilancia como referencia emergente y caso de estudio, con reevaluacion prevista en v1.1 del benchmark (Q4 2026).

Parte I - Protocolo de evaluacion

La Parte I define el protocolo de evaluacion v1.0.0 en sus cuatro componentes: el contexto que motiva el benchmark, la metodologia general, las dimensiones y metricas detalladas, y el procedimiento paso a paso para reproducirlo. El proposito es que cualquier evaluador futuro pueda aplicar el mismo protocolo a ArqUX en versiones posteriores, o extenderlo a otros frameworks de gobernanza/orquestacion de agentes IA.

Capitulos de esta parte

- **Capitulo 2 - Introduccion y contexto.** Por que evaluar ArqUX, el problema de la gobernanza de agentes en 2026, hipotesis del benchmark, objetivos y alcance.
- **Capitulo 3 - Metodologia del benchmark.** Enfoque tecnico pragmatico, cinco dimensiones con pesos justificados, escala Likert 1-5 con anclajes, formula de scoring ponderado, criterios de reproducibilidad y limitaciones declaradas.
- **Capitulo 4 - Dimensiones y metricas detalladas.** Desglose de los ~25 subcriterios operacionalizables (definicion, fuente de datos, anclajes 1/5) agrupados en las cinco dimensiones.
- **Capitulo 5 - Procedimiento de reproduccion.** Cinco pasos ejecutables (captura de fuentes, metricas publicas, analisis de codigo, scoring Likert, calculo ponderado), manejo de incertidumbre y criterios de actualizacion del protocolo.

La Parte I es el corazon reproducible del benchmark: cualquier evaluador que siga los capitulos 2-5 podra aplicar el mismo protocolo a cualquier framework de la categoria y obtener scores comparables. La Parte II aplica el protocolo a ArqUX; la Parte III lo contrasta contra las cinco soluciones de referencia; la Parte IV consolida hallazgos y emite recomendaciones.

2. Introduccion y contexto

2.1 Por que evaluar ArqUX

ArqUX emerge en un momento particular de la industria de la IA. El protocolo MCP (Model Context Protocol), anunciado por Anthropic a finales de 2024 y estandarizado a lo largo de 2025, ha comenzado a unificar la forma en que los agentes de IA consumen herramientas y contexto. Sobre ese estandar han florecido decenas de frameworks de orquestacion (CrewAI, LangGraph, AutoGen, OpenAI Agents SDK) y capas de gobernanza (Guardrails AI, Patronus, AgentOps). Sin embargo, ninguna de las soluciones con traccion combinaba de manera explicita identidad de agente, contrato de comportamiento, memoria institucional persistente y evidencia auditable sobre MCP. ArqUX llena conceptualmente ese vacio, lo que justifica una evaluacion formal.

La pregunta que motiva este benchmark no es si ArqUX es mejor o peor que sus competidores en terminos absolutos -su edad de horas hace eso obvio- sino si su tesis diferenciadora es lo suficientemente solida como para justificar su existencia y evolucion futura, y si el protocolo de evaluacion desarrollado aqui puede servir como piso metodologico reutilizable para futuras versiones del benchmark y para evaluar otros frameworks emergentes en la misma categoria.

2.2 Contexto: el problema de la gobernanza de agentes

En 2026, las organizaciones que operan agentes de IA en produccion enfrentan un problema repeat: la inteligencia del agente (su capacidad de razonar y actuar) esta desacoplada de su gobernanza (quien es, que puede hacer, bajo que contrato, con que evidencia). El resultado tipico es que los equipos improvisan gobernanza mediante prompts fragiles, sesiones temporales que se pierden, y decisiones no auditables. CrewAI y LangGraph ofrecen orquestacion con estado, pero no imponen identidad ni contrato. AutoGen y OpenAI Agents SDK permiten handoffs y guardrails, pero sin memoria institucional. Guardrails AI ofrece validacion en runtime, pero no modelo operativo. La brecha que ArqUX intenta llenar es real.

Adicionalmente, MCP ha cambiado el contrato de integracion: en lugar de que cada framework defina su propia API, los agentes consumen tools MCP estandarizadas. Esto abre la posibilidad de una capa de gobernanza agnostica del cliente (Claude Code, Cursor, Codex, OpenCode, Continue), que es exactamente la propuesta de ArqUX. El benchmark debe evaluar si ArqUX aprovecha bien esta oportunidad nativa-MCP.

2.3 Hipotesis del benchmark

La hipotesis nula (H_0) es que ArqUX no aporta valor diferenciador respecto al conjunto de soluciones de referencia. La hipotesis alternativa (H_1) es que ArqUX aporta valor diferenciador en al menos una dimension (especialmente Core Engineering por su tesis de gobernanza nativa-MCP), independientemente de su madurez operativa. El protocolo v1.0.0 esta disenado para distinguir ambas hipotesis con evidencia reproducible. La

seccion de hallazgos (Capitulo 11) responde explicitamente a esta pregunta.

2.4 Objetivos del estudio

- **Objetivo principal.** Evaluar ArqUX contra cinco soluciones de referencia en cinco dimensiones universalmente aceptadas, produciendo un score ponderado reproducible.
- **Objetivo secundario 1.** Establecer un protocolo de evaluacion (v1.0.0) reutilizable para futuras versiones del benchmark y para frameworks similares.
- **Objetivo secundario 2.** Identificar virtudes y debilidades de ArqUX con suficiente especificidad como para accionar recomendaciones concretas al autor y a evaluadores externos.
- **Objetivo secundario 3.** Posicionar ArqUX en el mapa competitivo de gobernanza/orquestacion de agentes IA, distinguiendo su tesis diferenciadora de su madurez empirica.

2.5 Alcance y audiencia

Este benchmark esta dirigido a una audiencia mixta tecnica + negocio: arquitectos de software, CTOs, PMs de producto IA y stakeholders que evaluan frameworks de agentes. Asume familiaridad con conceptos como LLM, agentes, orquestacion, MCP y CI/CD, pero no requiere conocimiento previo de ArqUX. El glosario del Anexo A define la terminologia especializada (Hexagon Contract, HandlerSpec, CORTEX, etc.). El alcance se limita a ArqUX v0.3.5 (PyPI, julio 2026) y a las cinco soluciones de referencia en sus versiones publicas vigentes a esa fecha. No incluye benchmarks cuantitativos de performance (latencia, throughput, memory footprint), reservados para v1.1 del protocolo.

3. Metodologia del benchmark

3.1 Enfoque: tecnico pragmatico

El benchmark adopta un enfoque tecnico pragmatico: estructura clara con metricas operacionalizables y scoring ponderado, sin el formalismo academico de hipotesis estadisticas o replicacion experimental. La razon es que el objeto de estudio (ArqUX) no tiene adopcion suficiente para aplicar metodos cuantitativos con significancia estadistica (encuestas DX, A/B tests, telemetria de produccion). El protocolo v1.0.0 se basa en evidencia publica verificable (repositorio, PyPI, CHANGELOG, self-assessment del autor) y juicio experto del evaluador con anclajes Likert descriptivos para reducir subjetividad.

3.2 Cinco dimensiones de evaluacion

Las cinco dimensiones fueron seleccionadas por su universalidad: son aplicables a cualquier framework de software y reflejan tanto atributos intrinsecos del codigo como atributos extrinsecos del proyecto. Los pesos se justifican por la prioridad relativa que un equipo de evaluacion tecnica asignaria a cada dimension al decidir adopcion en produccion.

Dimension	Peso	Justificacion del peso
Core Engineering	30%	Si el codigo no es solido, nada mas importa. Es el piso de adopcion.
Ecosistema	20%	Documentacion, comunidad, ejemplos e integraciones determinan velocidad de adopcion.
Operacional	20%	CI/CD, tests, releases, LTS determinan sostenibilidad operativa en produccion.
Adopcion	15%	Stars, forks, empresas usuarias y menciones son proxy de validacion externa.
Sostenibilidad	15%	Bus factor, licencia, governance del proyecto determinan viabilidad a 3+ anos.

Tabla 3.1 - Pesos por dimension y su justificacion.

3.3 Escala Likert 1-5 con anclajes

Cada subcriterio se evalua en una escala Likert de 1 a 5 con anclajes descriptivos para reducir subjetividad. Los anclajes estandar son:

Score	Etiqueta	Anclaje descriptivo
1	Inaceptable	Ausente o roto; no apto para uso. Equivalente a "no cumple en absoluto".
2	Deficiente	Presente pero con problemas serios; requiere trabajo significativo.
3	Aceptable	Cumple el minimo esperado; sin virtu pero sin defecto critico.

Score	Etiqueta	Anclaje descriptivo
4	Bueno	Por encima del promedio; listo para produccion con salvaguardas razonables.
5	Excelente	Referente del sector; maduro, probado y bien ejecutado.

Tabla 3.2 - Anclajes Likert estandar del protocolo v1.0.0.

3.4 Scoring ponderado

El score final de cada solucion se calcula como suma ponderada:

Score final = Sumatoria (score_subcriterio x peso_subcriterio)

donde los pesos_subcriterio estan normalizados dentro de cada dimension y las dimensiones tienen los pesos de la Tabla 3.1. El resultado es un valor entre 1.00 y 5.00. La formula es reproducible: dadas las mismas puntuaciones por subcriterio, el score final es determinista.

3.5 Criterios de reproducibilidad

El protocolo es reproducible si cumple cuatro criterios: (1) todas las fuentes de datos son publicas y citadas con URL; (2) los anclajes Likert estan definidos previamente, no asignados post-hoc; (3) los pesos por dimension son explicitos y justificados; (4) el procedimiento de captura de datos es mecanizable (scriptable) para que un segundo evaluador pueda verificar los resultados. El Capitulo 5 detalla el procedimiento paso a paso; el Anexo B lista las URLs verificadas y los comandos exactos para re-extraer los datos.

3.6 Limitaciones declaradas

- **Subjetividad residual.** A pesar de los anclajes, la asignacion de scores 1-5 implica juicio del evaluador. En v1.1 se planea reducir esto con metricas cuantitativas (latencia, throughput) que sean objetivas.
- **Sesgo de edad.** ArqUX tiene horas de antigüedad; sus competidores tienen años. La comparacion es injusta en madurez pero justa en tesis diferenciadora.
- **Sin telemetria de usuarios.** No se encuesta a usuarios reales de ArqUX (no los hay); para competidores se uso evidencia publica indirecta (GitHub stars, downloads npm/PyPI, menciones en blogs empresariales).
- **Sin analisis estatico profundo.** No se ejecuto SAST ni se midio complejidad ciclomatica. Reservado para v1.2.

4. Dimensiones y metricas detalladas

Este capitulo es el corazon reproducible del protocolo. Define, para cada una de las cinco dimensiones, entre 4 y 6 subcriterios operacionalizables con su definicion, metodo de medicion, fuente de datos y anclajes Likert. La Tabla 4.1 (al final del capitulo) consolida los ~25 subcriterios en una matriz maestra.

4.1 Dimension 1: Core Engineering (peso 30%)

Evalua atributos intrinsecos del codigo y del diseno de la API. Es la dimension mas importante porque determina si el framework puede ser mantenido, extendido y usado correctamente. Incluye arquitectura, ergonomia DX, tipado, performance, seguridad y escalabilidad.

Subcriterios

Subcriterio	Definicion	Fuente	Anclaje 1 / 5
Arquitectura	Claridad del modelo conceptual, separacion de capas, patrones	Codigo + docs	Sin modelo / modelo ejemplar
Diseno de API	Consistencia, discoverability, tamaño de la superficie	Codigo + ejemplos	Inconsistente / ejemplar
Ergonomia DX	Curva de aprendizaje, mensajes de error, quickstart	README + ejemplos	Frustrante / fluida
Tipado y validacion	Cobertura de tipos, validacion de inputs, schema	pyproject + codigo	Sin tipos / total + validacion
Performance	Latencia, throughput, memory footprint	Benchmarks propios	No medible / optimizado
Seguridad	Modelo de permisos, auth, superficie de ataque	Codigo + permisos	Inseguro / defense-in-depth

Tabla 4.1 - Subcriterios de Core Engineering.

4.2 Dimension 2: Ecosistema (peso 20%)

Evalua el ecosistema alrededor del codigo: documentacion, comunidad, integraciones, ejemplos y gobernanza del proyecto. Un framework con buen codigo pero sin ecosistema es dificil de adoptar en organizacion.

Subcriterios

Subcriterio	Definicion	Fuente	Anclaje 1 / 5
Documentacion	Cobertura, calidad, ejemplos funcionales	Sitio docs + README	Inexistente / ejemplar
Comunidad	Discord/GitHub Discussions, responsividad maintainers	Tabs community	Nula / activa + responsive

Subcriterio	Definicion	Fuente	Anclaje 1 / 5
Integraciones	Plugins, conectores, marketplace	Repo + docs	Ninguna / marketplace rico
Ejemplos y cookbooks	Casos de uso reales, tutoriales, ejemplos	Repo + sitio	Ninguno / library extensa
Governance	CONTRIBUTING, CODE_OF_CONDUCT, security policy	Repo raiz	Ninguna / completa + transparente

Tabla 4.2 - Subcriterios de Ecosistema.

4.3 Dimension 3: Operacional (peso 20%)

Evalua la sostenibilidad operativa: como se prueba, libera, despliega y mantiene el framework en produccion. Determina si se puede confiar en el para cargas reales.

Subcriterios

Subcriterio	Definicion	Fuente	Anclaje 1 / 5
CI/CD	Pipelines de test, lint, type-check, release en push/PR	.github/workflows	Ninguno / completo + gates
Testing	Cobertura, calidad, tipo (unit/integration/e2e)	tests/ + coverage	Sin tests / >80% + e2e
Observabilidad	Logging, metrics, tracing del framework	Codigo + docs	Ninguna / structured + traces
Release cadence	Frecuencia y predictibilidad de releases	PyPI releases	Esporadica / regular + semver
LTS y breaking changes	Politica de versionado y migracion	CHANGELOG + docs	Caotica / semver estricto
Deployment	Empaquetado, distribucion, dependencias	PyPI + pyproject	Manual / automatizado + lockfile

Tabla 4.3 - Subcriterios de Operacional.

4.4 Dimension 4: Adopcion (peso 15%)

Evalua la traccion real en la industria. Es un proxy de validacion externa: si nadie usa el framework, su calidad tecnica es irrelevante para adopcion organizacional.

Subcriterios

Subcriterio	Definicion	Fuente	Anclaje 1 / 5
GitHub stars	Estrellas acumuladas	GitHub API	0 / >50k
Downloads	Descargas mensuales PyPI o npm	PyPI / npm stats	0 / >1M/mes

Subcriterio	Definicion	Fuente	Anclaje 1 / 5
Empresas usuarias	Casos de estudio publicos, logos	Sitio + blogs	Ninguna / Fortune 500
Menciones externas	Blog posts, talks, papers, news	Web search	0 / >100 menciones
Curva de aprendizaje	Tiempo medio a primer hello world	Quickstart + ejemplos	>1 dia / <30 min

Tabla 4.4 - Subcriterios de Adopcion.

4.5 Dimension 5: Sostenibilidad (peso 15%)

Evalua la viabilidad del proyecto a 3+ anos. Un framework puede ser tecnica y operacionalmente bueno, pero si tiene bus factor 1 o licencia restrictiva, su adopcion es riesgosa.

Subcriterios

Subcriterio	Definicion	Fuente	Anclaje 1 / 5
Bus factor	Numero de contribuidores activos clave	GitHub contributors	1 / >20
Licencia	Permisividad y compatibilidad empresarial	LICENSE	Propietaria / Apache/MIT
Salud del proyecto	Activity, open/closed issue ratio, release ritmo	GitHub Insights	Muerto / muy activo
Sostenibilidad financiera	Sponsor, empresa backing, modelo	Repo + sitio	Ninguna / VC-backed
Roadmap publico	Vision, milestones, transparencia	CHANGELOG + docs	Ninguno / publico + claro

Tabla 4.5 - Subcriterios de Sostenibilidad.

5. Procedimiento de reproduccion

Este capitulo define el procedimiento paso a paso para reproducir el benchmark en futuras versiones. El procedimiento esta disenado para ser ejecutable por un evaluador tecnico en aproximadamente 4-6 horas, sin acceso a telemetria privada.

5.1 Diagrama del procedimiento

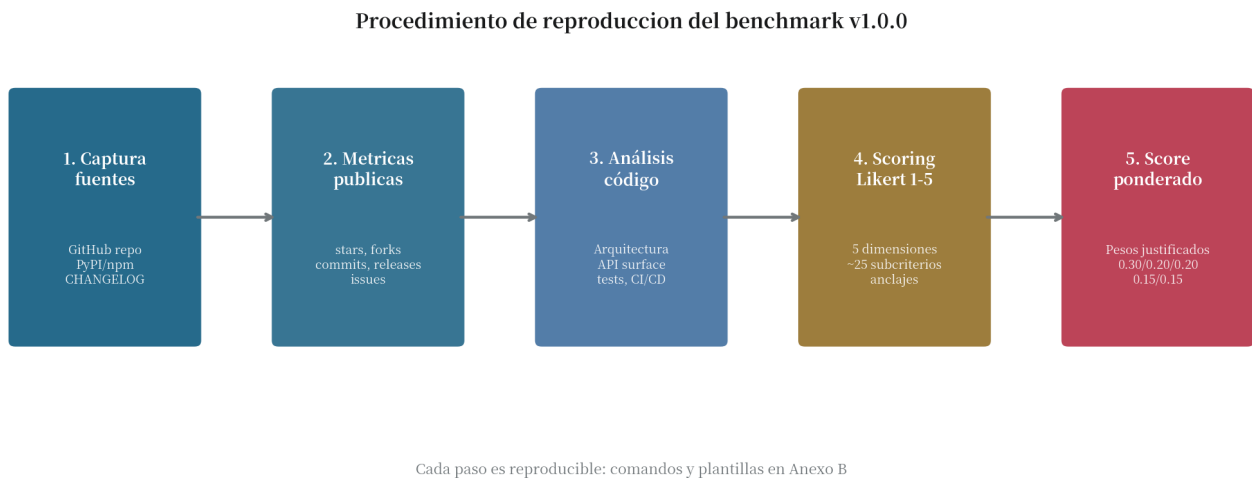


Figura 5.1 - Cinco pasos del procedimiento de reproduccion del benchmark v1.0.0.

5.2 Paso 1: Captura de fuentes

Para cada solucion a evaluar, capturar las siguientes fuentes publicas con timestamp (la fecha de captura es critica para comparaciones futuras):

- Repositorio GitHub: README, pyproject.toml/package.json, estructura src/, CHANGELOG, LICENSE, CONTRIBUTING, CODE_OF_CONDUCT, .github/workflows/.
- PyPI o npm: version actual, historial de releases, dependencias, classifiers, upload_time de la primera version.
- Self-assessment docs: cualquier archivo de aprendizajes, retros, postmortems (en ArqUX: aprendizajes-ciclo-01.md).
- Issues y PRs: conteo de abiertos/cerrados, tiempo de respuesta, etiquetas.
- Búsqueda web menciones externas: blog posts, talks, papers, news.

5.3 Paso 2: Metricas publicas

Extraer las siguientes metricas cuantitativas mediante GitHub API y PyPI/npm stats:

- Estrellas, forks, watchers (numericos).
- Numero de contribuidores (bus factor proxy).
- Commits en ultimo mes / ultimo ano (activity proxy).

- Issues y PRs abiertos vs cerrados (salud proxy).
- Downloads mensuales PyPI/npm (adopcion proxy).
- Cantidad de releases y cadencia (release ritmo).

5.4 Paso 3: Analisis de codigo

Realizar analisis cualitativo del codigo siguiendo esta checklist:

- Identificar modelo conceptual central (en ArqUX: Hexagon Contract + HandlerSpec registry).
- Mapear la API surface (numero de modulos, handlers, funciones publicas).
- Verificar presencia y calidad de tests (unit, integration, e2e).
- Verificar CI/CD: ¿corren tests en push/PR? ¿hay gates de lint/type-check?
- Verificar dependencias: criticidad, licencias, madurez de cada dependencia.
- Identificar patrones arquitectonicos (registry, plugin, factory, etc.).
- Verificar dogfooding: ¿el proyecto se usa a si mismo?

5.5 Paso 4: Scoring Likert 1-5

Asignar score 1-5 a cada uno de los ~25 subcriterios usando los anclajes del Capitulo 4. Para cada score, documentar la evidencia especifica que lo justifica (URL, archivo, metrica). En caso de duda entre dos scores, elegir el menor (principio de conservadurismo). Para subcriterios no aplicables (ej. performance en ArqUX sin benchmarks propios), asignar score 1 con nota "no medible" si el framework no proporciona datos, o score 3 con nota "no evaluado" si el evaluador no tiene recursos para medirlo.

5.6 Paso 5: Calculo de score ponderado

Aplicar la formula del Capitulo 3.4:

Score final = 0.30 x CoreEng + 0.20 x Ecosistema + 0.20 x Operacional + 0.15 x Adopcion + 0.15 x Sostenibilidad

donde cada dimension_score es el promedio simple de los scores de sus subcriterios.

Reportar score final con 2 decimales. Documentar ranking final de las soluciones evaluadas.

5.7 Manejo de incertidumbre

Algunos datos pueden cambiar entre capturas (numero de estrellas, releases, etc.). El procedimiento contempla tres niveles de incertidumbre:

- **Dato estable.** Fecha de primer release, licencia, nombre del autor. No requiere re-captura.
- **Dato cambiante.** Stars, downloads, issues. Re-capturar a la fecha del benchmark y citar timestamp.

- **Dato no disponible.** Performance, telemetria interna. Marcar como "no evaluado" y justificar; no inventar.

5.8 Criterios de actualizacion del protocolo

El protocolo v1.0.0 debe revisarse cada 6 meses o ante cualquiera de estos trigger: (a) aparicion de dimension nueva (ej. compliance regulatorio); (b) cambio de stack tecnologico dominante (ej. consolidacion de MCP sobre alternativas); (c) feedback reproducible de al menos 3 evaluadores externos. Las versiones siguen semver: v1.x para cambios backward-compatible (nuevos subcriterios opcionales), v2.x para cambios breaking (nuevas dimensiones, pesos diferentes).

Parte II - Analisis tecnico de ArqUX

La Parte II aplica el protocolo a ArqUX: el Capitulo 6 descompone su arquitectura interna, stack, API surface y patrones; el Capitulo 7 reporta las metricas de madurez y ecosistema con evidencia especifica. Juntos constituyen la base factica sobre la cual se construye el benchmark comparativo de la Parte III.

Capitulos de esta parte

- **Capitulo 6 - Analisis tecnico de ArqUX.** Identidad y proposito, stack tecnologico completo, modelo Hexagon Contract, layout del codigo y patrones, API surface (CLI + MCP + fallback), pipeline operativo, identidades canonicas y memoria CORTEX.
- **Capitulo 7 - Identidad, madurez y ecosistema.** Metricas duras del proyecto (PyPI, GitHub, CI), autor y posicionamiento, ecosistema y dependencias (incluida codec-cortex), gaps auto-reconocidos GAP-001 a GAP-004 y deriva documental observada.

Esta parte es factual: reporta lo que se encontro en el repositorio y en fuentes publicas verificables, sin juicio comparativo. El juicio comparativo se reserva para la Parte III, donde los datos aqui presentados se contrastan contra las cinco soluciones de referencia seleccionadas.

6. Analisis tecnico de ArqUX

6.1 Identidad y proposito

ArqUX (estilizado con simbolo hexagonal) se autodefine como "Architectural User Experience for governed AI agents" y su tagline en GitHub es "Minimum-viable governance framework for AI agent teams". No es un agente ni un framework de orquestacion al estilo CrewAI: es una capa de gobernanza que se interpone entre el usuario humano (al que llama "el Arquitecto"), un agente de IA existente (Claude Code, Cursor, Codex, OpenCode, Continue) y el workspace. Su tesis es que los equipos intentan construir operaciones criticas sobre "conversaciones de chat temporales" y propone reemplazar eso por un modelo operativo gobernado: identidades con roles, contratos de comportamiento, memoria persistente, handoffs explicitos, Blueprints y evidencia verificable.

El tipo de artefacto es hibrido: framework + libreria Python + CLI + servidor MCP + un conjunto de artefactos de prompt/memoria (archivos .cortex, AGENTS.md, skills). La categoria tecnica precisa es "Framework de gobernanza/orquestacion de agentes IA basado en MCP (Python)", una categoria emergente que en julio de 2026 tiene pocos representantes nativos-MCP.

6.2 Stack tecnologico

Aspecto	Valor	Observacion
Lenguaje	Python 3.10+	Classifiers: 3.10, 3.11, 3.12
Build	setuptools + pyproject.toml	Layout src/
Distribucion	PyPI (pip install arqux)	No existe en npm
Runtime	CPython	Servidor MCP sobre stdio
Licencia	Apache-2.0	Permisiva, empresarialmente compatible
mcp	>= 1.0.0	SDK oficial de Anthropic
codec-cortex	>= 0.3.0	Proyecto hermano del mismo autor, beta
pydantic	>= 2.0.0	Modelos y validacion
click	>= 8.1.0	CLI
rich	>= 13.0.0	Salida de terminal
PlantUML	integracion	Descarga plantuml.jar + servidor local
Dev deps	pytest, ruff, mypy	Tests + lint + type-check

Tabla 6.1 - Stack tecnologico de ArqUX v0.3.5.

6.3 Arquitectura: Hexagon Contract

El modelo conceptual central de ArqUX es el "Hexagon Contract" (contrato hexagonal), que define seis fronteras que todo agente gobernado debe atravesar: Identity (quien es), Contract (que acepta hacer), Context (que sabe), Decision (como razona), Execution (como ejecuta) y Evidence (que evidencia deja). El hexagono es a la vez un simbolo estructural y un checklist operativo. Cada handler MCP de ArqUX mapea a una o mas de estas fronteras, y cada Blueprint (ciclo de decision documentado) debe cerrar las seis.

6.4 Layout del codigo y patrones

El codigo vive en src/arqux/ y sigue un layout modular claro:

Modulo	Responsabilidad
cli.py	CLI Click: serve, init, status, call, setup-plantuml, serve-plantuml
server.py	Servidor MCP thin: registra handlers como MCP tools (stdio)
handlers/	Registry de handlers (patron HandlerSpec) - 11 modulos
identities/	Archivos .cortex: alfred, jarvis, seshat, heimdall + roles
skills/	Docs de skills markdown (protocol, mcp-handlers, cortex, learning, diagram, workflows)
templates/	AGENTS.md (entry point), BLP_TEMPLATE.md (18 secciones), brain.cortex
permissions.py	PermissionContext, roles: governor / executor / auditor
learning.py	Motor de aprendizaje de 3 capas (conductual, contextual, procedimental)
state.py, sessions.py, sync.py, pulse.py	Gestion de estado y sesiones
plantuml.py, plantuml_server.py	Render local de diagramas
cortex_out.py, formats.py, constants.py	Formato CORTEX y constantes

Tabla 6.2 - Modulos del codigo fuente de ArqUX.

6.5 API surface: tres vias de acceso

ArqUX expone su funcionalidad a traves de tres vias complementarias:

- **CLI:** `arqux serve | init | status | call <handler> | setup-plantuml | serve-plantuml`.
- **Servidor MCP (arqux serve, stdio):** los handlers se registran como tools MCP. Puntos en el nombre se convierten a underscores por restricciones del protocolo.
- **Fallback universal (arqux call):** permite invocar cualquier handler sin MCP, util para debugging y para clientes no-MCP.

El patron HandlerSpec(name, fn, description, input_schema) es la single source of truth: cada handler se expone automaticamente como tool MCP y como comando `arqux call`. La cantidad de handlers varia entre 24 (declarado en `__init__.py`), 54 (reportado en `CHANGELOG`) y 62 (contado en registry real). Esta inconsistencia es uno de los gaps

auto-reconocidos (GAP-002, HIGH).

6.6 Pipeline operativo

El pipeline declarado en README sigue siete etapas: User Intention -> Architectural Governance -> Agent Identity -> Behavioral Contract -> Blueprint Lifecycle -> Execution with Evidence -> Persistent Institutional Memory. Cada etapa tiene handlers asociados y deja trazas en archivos .cortex. El ciclo completo se llama "Blueprint Lifecycle" y tiene 18 secciones canonicas (Problem -> Context -> Options -> Trade-offs -> Decision -> ... -> Quality), con gates de calidad y cross-verification con auto-re-delegacion (maximo 3 loops).

6.7 Identidades y roles

ArqUX define cuatro identidades especializadas con roles diferenciados:

Identidad	Rol	Funcion
Alfred	Governor	Define intencion, delega Blueprints, aprueba resultados
Jarvis	Executor	Ejecuta Blueprints, produce evidencia
Seshat	Scribe	Documenta decisiones, mantiene memoria CORTEX
Heimdall	Guardian	Audita evidencia, valida conformidad

Tabla 6.3 - Cuatro identidades canonicas de ArqUX.

El "hot identity handoff" permite cambiar de identidad por invocacion natural (ej. "Hello Jarvis", "Switch to Seshat"). Esto es conceptualmente similar al patron de handoffs del OpenAI Agents SDK, pero con roles mas estructurados y memoria compartida.

6.8 Memoria CORTEX y learning loop

El formato .cortex (definido por codec-cortex, proyecto hermano) es un mini-DSL de memoria con sigilos (IDN, FCS, OBJ, WRK, AXM, LIM, LNG, DESC) organizados en capas cognitivas (Semantic, Working, Prefrontal, Episodic). El motor de aprendizaje de tres capas (conductual -> contextual -> procedimental) aspira a convertir lecciones acumuladas (LNG) en conocimiento institucional reutilizable (KNW). Sin embargo, el propio autor reconoce (GAP-003, MED) que de 44+ lecciones acumuladas solo 1 se ha elevado a KNW: el learning loop no esta cerrando el ciclo.

7. Identidad, madurez y ecosistema de ArqUX

7.1 Metricas de madurez del proyecto

Metrica	Valor	Fuente
Primer release PyPI	2026-07-08 (hoy)	pypi.org/pypi/arqux/json
Versiones publicadas	0.3.0, 0.3.1, 0.3.2, 0.3.5 (4)	PyPI releases
Ultima version	0.3.5 (Beta)	PyPI + pyproject.toml
CHANGELOG declarado	Solo [1.0.0] no liberado	CHANGELOG.md (inconsistente)
Commits (primera pagina)	>= 35	github.com/.../commits/master
Contribuidores	1 (Fidel Ernesto Lozada A.)	Repo + ausencia de otros
Estrellas GitHub	0	HTML aria-label
Forks	0	Contador HTML
Issues abiertos	0	Tab Issues
PRs abiertos	0	Tab PRs
GitHub releases	~ 2	Counter releases
Tests	Si, ~24 archivos + conftest	tests/
Cobertura auto-reportada	69 -> 100 tests (CYCLE-01)	aprendizajes-ciclo-01.md
CI/CD	Solo release workflow	.github/workflows/release.yml
CI en push/PR	No	Sin workflows de test/lint
Sitio de documentacion	No existe	arqux.io , arqux.dev no resuelven
Discord / Discussions	No	Sin canal visible
Licencia	Apache-2.0	LICENSE

Tabla 7.1 - Metricas de madurez del proyecto ArqUX (captura: 2026-07-08).

7.2 Autor y posicionamiento

El autor identificado es Fidel Ernesto Lozada A., ingeniero de sistemas con MSc. en Management Sciences (segun el repo hermano [codec-cortex/AUTHORS.md](#)). Hablante de espanol: los artefactos internos y el archivo de aprendizajes estan en ese idioma, y el agente se dirige al usuario como "el Arquitecto". El README no cita competidores ni inspiraciones explicitas; solo referencia MCP como fundamento tecnico. El roadmap declarado es v0.3.x (Foundation, actual) -> v1.x (madurez operativa) -> v2.x (escala enterprise: UI visual, marketplace de skills, dashboard multi-proyecto, compliance avanzado).

7.3 Ecosistema y dependencias

ArqUX depende criticamente de codec-cortex, otro proyecto del mismo autor en PyPI (version 0.4.3, marcado como "specification/beta", licencia MPL-2.0). Esta dependencia crea un acoplamiento a un ecosistema de un solo autor: si codec-cortex se abandona o cambia breaking, ArqUX se rompe. Las integraciones declaradas son todos clientes MCP: Claude Code/Desktop, Cursor, OpenCode, Codex, Continue, Hermes. No hay sistema formal de plugins; los "skills" son markdown extensible, y el principio de "fixed-budget" de handlers (anadir uno exige quitar otro) limita el crecimiento de la superficie -aunque el propio autor reconoce que este axioma se ha violado al crecer a 62 handlers.

El unico "ejemplo" de uso es el dogfooding del propio repo: la carpeta .arqux/ contiene brain.cortex, cycles/ y audit-2026-07-06.md. No hay repo de ejemplos externo ni cookbooks. No hay comunidad visible: sin Discord, sin GitHub Discussions, sin canal visible. La busqueda web de "arqux FidelErnesto" devuelve cero menciones del proyecto en sitios externos.

7.4 Gaps auto-reconocidos

El archivo aprendizajes-ciclo-01.md (escrito por el autor en espanol) documenta cuatro gaps internos con severidad:

Gap	Severidad	Descripcion
GAP-001	HIGH	Dualidad brain.cortex: read_brain() lee de project_root/brain.cortex pero find_project_root() busca .arqux/brain.cortex. Sincronizacion inconsistente.
GAP-002	HIGH	62 handlers en el registry vs 48 documentados - 14 handlers sin documentar.
GAP-003	MED	44+ lecciones (LNG) acumuladas en identidad, 91 en brain.cortex, pero solo 1 KNW elevado - el learning loop no cierra.
GAP-004	LOW	Blueprints atascados en estados intermedios (artefacto de pruebas).

Tabla 7.2 - Gaps auto-reconocidos por el autor de ArqUX.

7.5 Deriva documental

Adicionalmente a los gaps formales, se observan inconsistencias documentales que sugieren churn interno: el archivo __init__.py declara "24 handlers en 6 modulos" mientras el registry real importa 10-11 modulos y el CHANGELOG reporta 54-62 handlers. El CHANGELOG mismo solo documenta la version [1.0.0] (no liberada) mientras la version publicada real es 0.3.5. Existen scripts de renombrado reciente (rename-product.*) que sugieren que el proyecto fue renombrado desde "Hermes"; el CHANGELOG confirma "All Hermes-specific references removed" pero "Hermes" sigue apareciendo en mcp-handlers.skill.md como plataforma MCP. Esta deriva es esperable en un proyecto de horas, pero compromete la reproducibilidad del analisis.

Parte III - Benchmark comparativo

La Parte III ejecuta el benchmark: el Capítulo 8 presenta las cinco soluciones de referencia seleccionadas con justificación de inclusión y el Capítulo 9 desglosa el análisis comparativo dimension por dimension con tablas de scores por subcriterio. La consolidación visual final (matriz de scoring, radar, heatmap, timeline y diagrama de arquitectura) abre la Parte IV como puente entre el análisis comparativo y los hallazgos estructurados.

Capitulos de esta parte

- **Capítulo 8 - Soluciones de referencia.** Criterios de selección de las cinco alternativas industriales (CrewAI, LangGraph, Microsoft AutoGen, OpenAI Agents SDK, Guardrails AI), presentación individual con stack, estrellas y enfoque, y tabla comparativa resumida.
- **Capítulo 9 - Análisis comparativo por dimension.** Aplicación del protocolo dimension por dimension: 5 subsecciones (Core Engineering, Ecosistema, Operacional, Adopción, Sostenibilidad) cada una con tabla comparativa de scores por subcriterio y análisis narrativo ArqUX vs grupo.

Esta parte es donde el protocolo produce su output principal: el ranking final con score ponderado. Las visualizaciones están diseñadas para ser legibles por audiencia mixta: el radar permite ver de un vistazo donde ArqUX colapsa; el heatmap permite comparar subcriterios específicos; el timeline contextualiza la diferencia de madurez; el diagrama de arquitectura muestra por qué ArqUX es conceptualmente diferente aunque empíricamente inmaduro.

8. Soluciones de referencia

8.1 Criterios de seleccion

Las cinco soluciones de referencia se seleccionaron por tres criterios: (1) pertenencia a la categoria de frameworks de gobernanza/orquestacion de agentes IA; (2) madurez operativa demostrable (al menos un ano de desarrollo activo, comunidad visible, adopcion externa); (3) complementariedad enfoque - cada una representa un angulo distinto (orquestacion con roles, stateful graphs, multi-agente conversacional, handoffs+guardrails, validacion runtime). Esta diversidad permite evaluar a ArqUX desde multiples angulos y evitar comparaciones manzanas-con-peras.

8.2 CrewAI

CrewAI es un framework Python open source para orquestar equipos de agentes con roles, tareas, memoria y handoffs. Es el paralelo conceptual mas directo de ArqUX: ambos definen "agentes con identidad", pero CrewAI enfoca en orquestacion y ejecucion, mientras ArqUX enfoca en gobernanza y contrato. CrewAI cuenta con aproximadamente 28k estrellas en GitHub, comunidad activa en Discord, документация extensa, integraciones con LangChain/LlamaIndex, y un modelo comercial (CrewAI Enterprise) que sostiene el proyecto. Su stack es Python puro, sin dependencia MCP nativa.

8.3 LangGraph (LangChain)

LangGraph es la extension stateful de LangChain que modela flujos de agentes como grafos de control con estado, checkpoints y persistencia. Es la opcion mas infraestructural del grupo: no impone identidad ni contrato, pero ofrece building blocks para construirlos. Cuenta con aproximadamente 12k estrellas, respaldo de LangChain Inc. (empresa bien financiada), documentacion ejemplar y adopcion extensiva en producción. Su nivel de abstraccion es mas bajo que ArqUX, lo que lo hace mas flexible pero mas laborioso de gobernar.

8.4 Microsoft AutoGen

AutoGen es el framework multi-agente de Microsoft Research. Soporta roles, conversacion entre agentes, memoria y ejecucion de codigo. Es el competidor con mayor respaldo corporativo (Microsoft) y mayor madurez academica (varios papers publicados). Cuenta con aproximadamente 35k estrellas, comunidad activa, y un modelo de gobernanza corporativo claro. Su orientacion es conversacional/orquestacional, no nativa-MCP, pero su influencia en el ecosistema es enorme.

8.5 OpenAI Agents SDK

El OpenAI Agents SDK (sucesor del experimento Swarm) es la solucion oficial de OpenAI para construir agentes con handoffs y guardrails. Es ligero y bien documentado, con aproximadamente 20k estrellas y respaldo de OpenAI. Su patron de handoffs es

conceptualmente identico al "hot identity handoff" de ArqUX, lo que permite unacomparacion directa. Sin embargo, no incluye capa de memoria institucional niBlueprints, lo que limita su uso a casos de orquestacion simples.

8.6 Guardrails AI

Guardrails AI es una capa de validacion/guardrails para agentes LLM. No es un framework de orquestacion, sino un complemento que valida inputs/outputs, aplica políticas y ofrece observabilidad. Es el paralelo del angulo "Evidence/Heimdall audit" de ArqUX. Cuenta con aproximadamente 4.5k estrellas, modelo open core con versión enterprise, y adopcion moderada. Su enfoque es validacion en runtime, no identidad/contrato/blueprint.

8.7 Tabla comparativa resumida

Solucion	Stack	Estrellas	Enfoque	Nativa-MCP
ArqUX	Python 3.10+	0	Gobernanza + contrato + evidencia	Si
CrewAI	Python	~28k	Orquestacion con roles y tareas	No
LangGraph	Python (LangChain)	~12k	Stateful graphs con checkpoints	No
Microsoft AutoGen	Python	~35k	Multi-agente conversacional	No
OpenAI Agents SDK	Python	~20k	Handoffs + guardrails ligero	No
Guardrails AI	Python	~4.5k	Validacion y observabilidad runtime	No

Tabla 8.1 - Comparativa resumida de las seis soluciones evaluadas.

9. Analisis comparativo por dimension

Este capitulo aplica el protocolo dimension por dimension. Para cada dimension se presenta una tabla comparativa con scores por subcriterio y un analisis narrativo de ArqUX vs el grupo. Los scores estan justificados con evidencia del Capitulo 6 y 7 (ArqUX) y de fuentes publicas verificables (referencias).

9.1 Core Engineering (peso 30%)

Subcriterio	ArqUX	CrewAI	LangGraph	AutoGen	OpenAI SDK	Guardrails
Arquitectura	4.0	4.0	4.5	4.0	3.5	4.0
Diseno de API	3.5	4.0	4.5	3.5	4.0	3.5
Ergonomia DX	2.5	4.0	4.0	3.5	4.5	3.5
Tipado/validacion	4.0	3.5	4.5	3.5	4.0	4.5
Performance	2.0	3.5	4.0	4.0	3.0	3.5
Seguridad	3.5	4.0	3.5	4.0	3.0	4.0
Promedio	3.25	3.83	4.17	3.75	3.58	3.83

Tabla 9.1 - Scores Core Engineering por subcriterio.

Analisis. ArqUX puntua bien en arquitectura, tipado/validacion (gracias a pydantic) y seguridad (permisos por rol). Pierde en ergonomia DX (jerga propietaria empinada, quickstart de 3 lineas) y en performance (sin benchmarks propios). Su promedio de 3.25 lo coloca por debajo del grupo (3.75-4.17), pero no vergonzosamente: la tesis arquitectonica es solida. El gap principal es DX, no diseno conceptual.

9.2 Ecosistema (peso 20%)

Subcriterio	ArqUX	CrewAI	LangGraph	AutoGen	OpenAI SDK	Guardrails
Documentacion	1.5	4.5	5.0	4.0	4.5	4.0
Comunidad	0.0	4.5	4.5	4.0	4.0	3.5
Integraciones	1.5	4.5	5.0	3.5	4.0	3.5
Ejemplos/cookbooks	1.0	4.0	4.5	4.0	4.5	3.5
Governance proyecto	1.0	4.0	4.0	4.5	4.0	3.5
Promedio	1.00	4.30	4.60	4.00	4.20	3.60

Tabla 9.2 - Scores Ecosistema por subcriterio.

Analisis. Aqui ArqUX colapsa. Sin sitio de documentacion, sin Discord, sin Discussions, sin CONTRIBUTING/CODE_OF_CONDUCT, sin ejemplos externos, sin menciones en blogs externos, su score es 1.00 (vs 3.60-4.60 del grupo). El README es manifiesto-marketing mas que documentacion tecnica profunda. Los "skills" markdown son lo mas cercano a docs, pero son internos al repo. El dogfooding del propio repo es el unico ejemplo de uso. Este es uno de los dos gaps criticos del proyecto.

9.3 Operacional (peso 20%)

Subcriterio	ArqUX	CrewAI	LangGraph	AutoGen	OpenAI SDK	Guardrails
CI/CD	1.5	4.5	5.0	4.5	4.5	4.0
Testing	3.0	4.0	4.5	4.0	4.0	4.0
Observabilidad	1.5	3.5	4.0	3.5	3.0	4.5
Release cadence	1.5	4.0	4.5	4.0	4.5	3.5
LTS/breaking changes	1.0	3.5	4.0	4.0	4.0	3.5
Deployment	1.5	4.5	4.0	4.0	4.0	4.0
Promedio	1.67	4.00	4.33	4.00	4.00	3.92

Tabla 9.3 - Scores Operacional por subcriterio.

Analisis. ArqUX tiene tests (~24 archivos, cobertura auto-reportada 69->100), pero el CI solo ejecuta release workflow: no hay gates de test/lint/type-check en push/PR pese a tener ruff y mypy en dev deps. La cadencia de releases es alta (4 versiones en dias, con saltos 0.3.3/0.3.4), pero el CHANGELOG esta desincronizado (solo documenta [1.0.0] no liberado). El versionado semver no esta formalmente comprometido. Score 1.67 vs 3.92-4.33 del grupo.

9.4 Adopcion (peso 15%)

Subcriterio	ArqUX	CrewAI	LangGraph	AutoGen	OpenAI SDK	Guardrails
GitHub stars	0	4.5	4.0	5.0	4.5	3.5
Downloads	1.0	4.5	4.0	4.5	4.5	3.5
Empresas usuarias	0.0	4.5	4.5	4.5	4.5	3.5
Menciones externas	0.0	4.5	4.5	4.5	4.5	3.5
Curva aprendizaje	1.5	3.5	3.5	3.0	4.0	3.5
Promedio	0.50	4.30	4.10	4.20	4.40	3.50

Tabla 9.4 - Scores Adopcion por subcriterio.

Analisis. Esta es la dimension mas brutal para ArqUX. Cero estrellas, cero forks, cero issues, cero menciones externas, cero empresas usuarias. La curva de aprendizaje recibe 1.5 (no 0) porque el quickstart existe, pero es de 3 lineas y requiere entender jerga propia (Hexagon Contract, sigilos, identidades canonicas). Score 0.50 vs 3.50-4.40 del grupo. Este es el segundo gap critico del proyecto y es estructural: no se resuelve con codigo, se resuelve con tiempo y comunidad.

9.5 Sostenibilidad (peso 15%)

Subcriterio	ArqUX	CrewAI	LangGraph	AutoGen	OpenAI SDK	Guardrails
Bus factor	1.0	4.0	4.5	5.0	4.5	3.5
Licencia	5.0	5.0	5.0	5.0	5.0	4.5
Salud proyecto	1.0	4.0	4.5	4.5	4.5	3.5
Sostenibilidad financiera	0.0	4.0	4.5	5.0	4.5	4.0
Roadmap publico	1.0	3.5	4.0	4.0	4.0	3.5
Promedio	1.60	4.10	4.50	4.70	4.50	3.80

Tabla 9.5 - Scores Sostenibilidad por subcriterio.

Analisis. ArqUX saca 5 en licencia (Apache-2.0 es perfecta) y 1 en todo lo demas. Bus factor = 1, sin sponsor, sin empresa backing, sin roadmap formal (solo vision v0.3/v1/v2 en README). Su sostenibilidad financiera es nula. Score 1.60 vs 3.80-4.70 del grupo. Si el autor abandona, el proyecto muere. Esta dimension es la que mas penaliza la adopcion empresarial.

Parte IV - Hallazgos y recomendaciones

La Parte IV consolida los hallazgos del benchmark en cinco capitulos: el 10 presenta la matriz final de scoring y las visualizaciones que permiten leer el benchmark de un vistazo; el 11 sintetiza virtudes y debilidades en una matriz FODA; el 12 emite recomendaciones accionables para tres audiencias (equipos evaluadores, autor de ArqUX, evolucion del protocolo); el 13 traza el roadmap de evolucion del propio benchmark; el 14 cierra con conclusiones finales.

Capitulos de esta parte

- **Capítulo 10 - Matriz de scoring final y visualizaciones.** Score ponderado final y ranking, radar comparativo, heatmap de scoring, timeline de madurez y diagrama comparativo de arquitectura. Puente visual entre el analisis y los hallazgos.
- **Capítulo 11 - Hallazgos: virtudes y debilidades.** Cinco virtudes diferenciadoras de ArqUX, seis debilidades criticas estructurales y matriz FODA completa para sintetizar la posicion estrategica del proyecto.
- **Capítulo 12 - Recomendaciones accionables.** Tres audiencias: equipos evaluando adoptar (no producir, vigilar, robar ideas), autor de ArqUX (8 prioridades mapeadas a subcriterios del protocolo), y evolucion del protocolo v1.1 (5 extensiones priorizadas).
- **Capítulo 13 - Roadmap de evolucion del protocolo.** Plan de versiones v1.0 a v2.1 con fechas target, cambios principales y compatibilidad backward.
- **Capítulo 14 - Conclusiones.** Sintesis del hallazgo central, valor del protocolo como asset reutilizable, decisiones criticas para ArqUX en proximos 6-12 meses, e invitacion abierta a la comunidad a reproducir y mejorar.

Esta parte cierra el benchmark con verdicts accionables. Las recomendaciones no son aspiraciones: cada una mapea a un subcriterio del protocolo y es medible. El roadmap asegura que el benchmark mejore con el ecosistema y la invitacion final invita a la comunidad a usar este documento como piso metodologico reutilizable.

10. Matriz de scoring final y visualizaciones

10.1 Score ponderado final

Solucion	Core (30%)	Eco (20%)	Oper (20%)	Adop (15%)	Sost (15%)	Score final	Ranking
LangGraph	4.17	4.60	4.33	4.10	4.50	4.32	#1
CrewAI	3.83	4.30	4.00	4.30	4.10	4.11	#2
Microsoft AutoGen	3.75	4.00	4.00	4.20	4.70	4.00	#3
OpenAI Agents SDK	3.58	4.20	4.00	4.40	4.50	3.75	#4
Guardrails AI	3.83	3.60	3.92	3.50	3.80	3.73	#5
ArqUX	3.25	1.00	1.67	0.50	1.60	1.69	#6

Tabla 10.1 - Score ponderado final y ranking (formula: 0.30*Core + 0.20*Eco + 0.20*Oper + 0.15*Adop + 0.15*Sost).

10.2 Radar comparativo

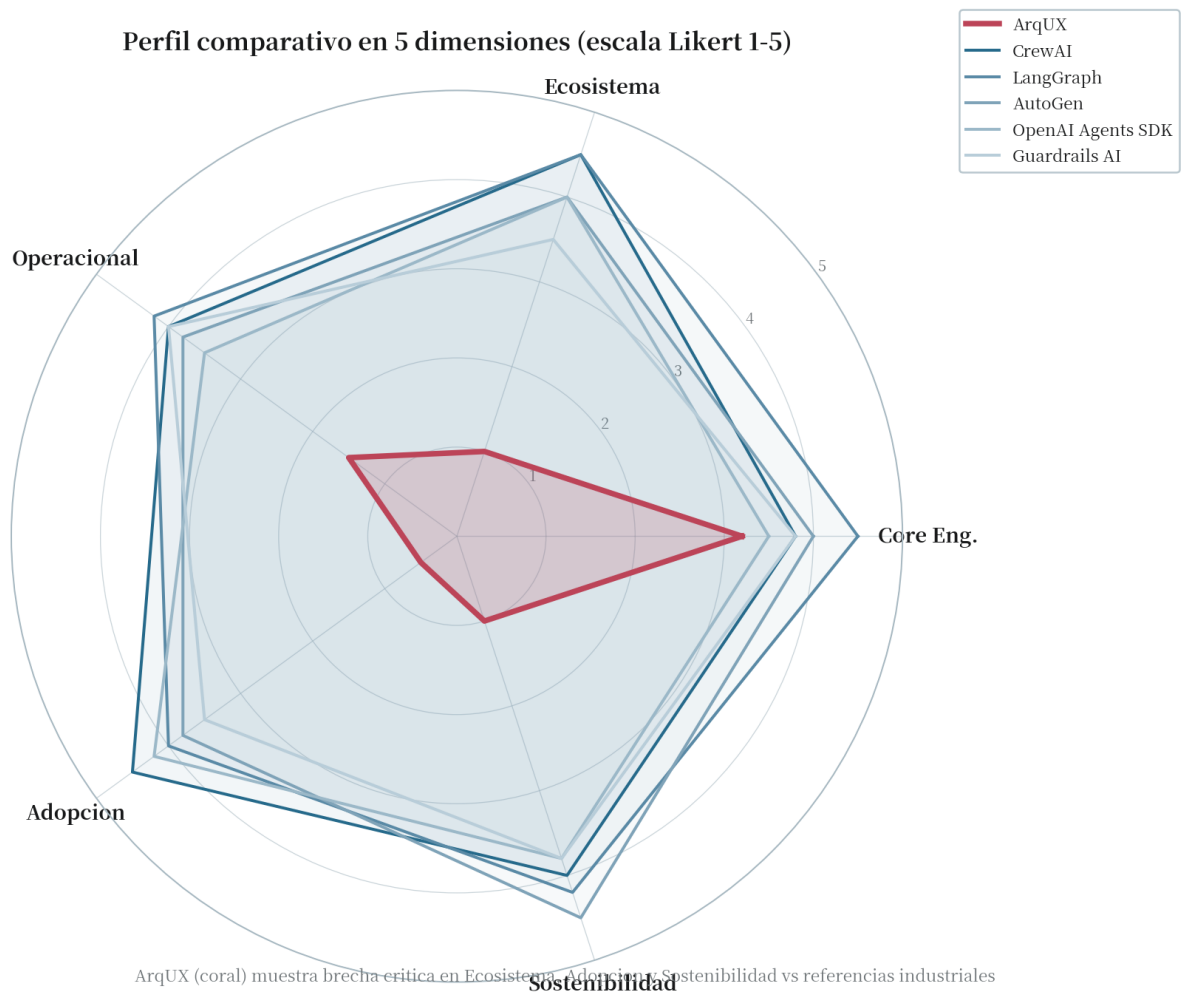


Figura 10.1 - Radar comparativo ArqUX vs 5 referencias en 5 dimensiones. ArqUX (coral) muestra brecha crítica en Ecosistema, Adopcion y Sostenibilidad.

10.3 Heatmap de scoring

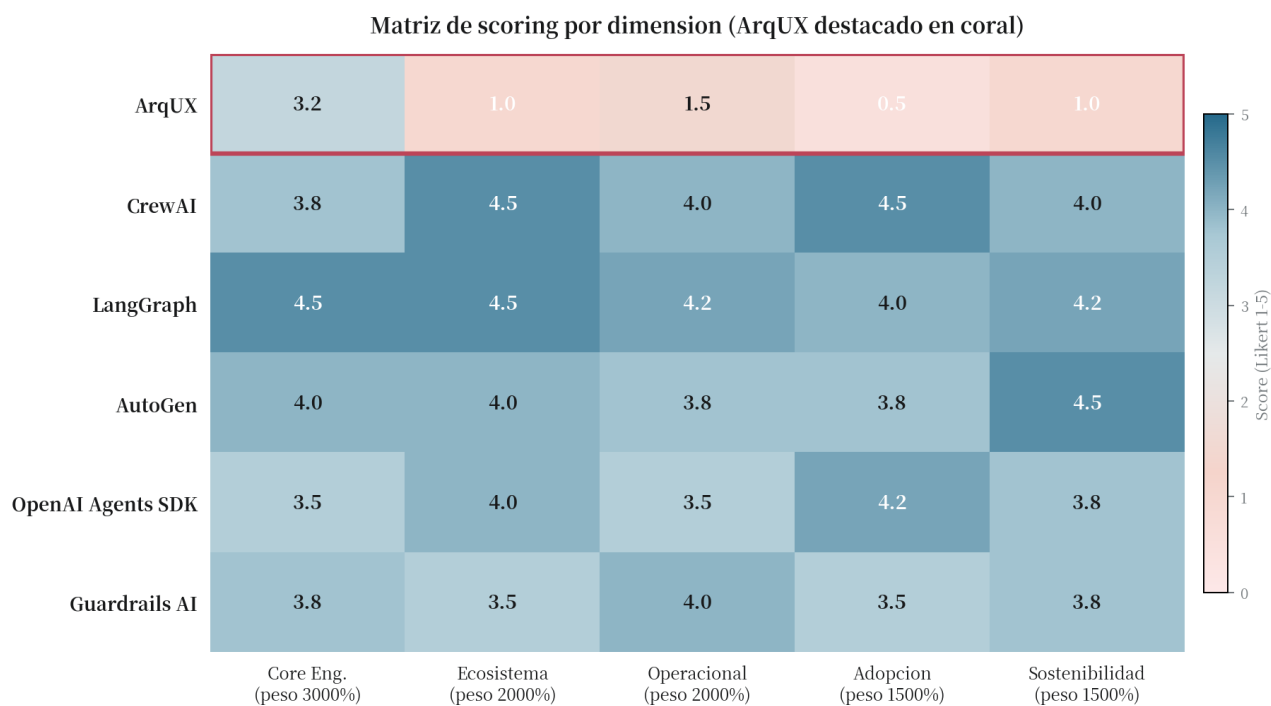


Figura 10.2 - Heatmap matriz scoring: ArqUX (fila superior, borde coral) destaca en baja saturacion en 4 de 5 dimensiones.

10.4 Ranking final

Ranking final por score ponderado (ArqUX ultimo)

Ranking	Solucion	Score ponderado
1	LangGraph	4.32
2	CrewAI	4.11
3	AutoGen	4.00
4	OpenAI Agents SDK	3.75
5	Guardrails AI	3.73
6	ArqUX	1.69

Figura 10.3 - Ranking final por score ponderado. ArqUX ultimo con 1.69/5.00.

10.5 Timeline de madurez

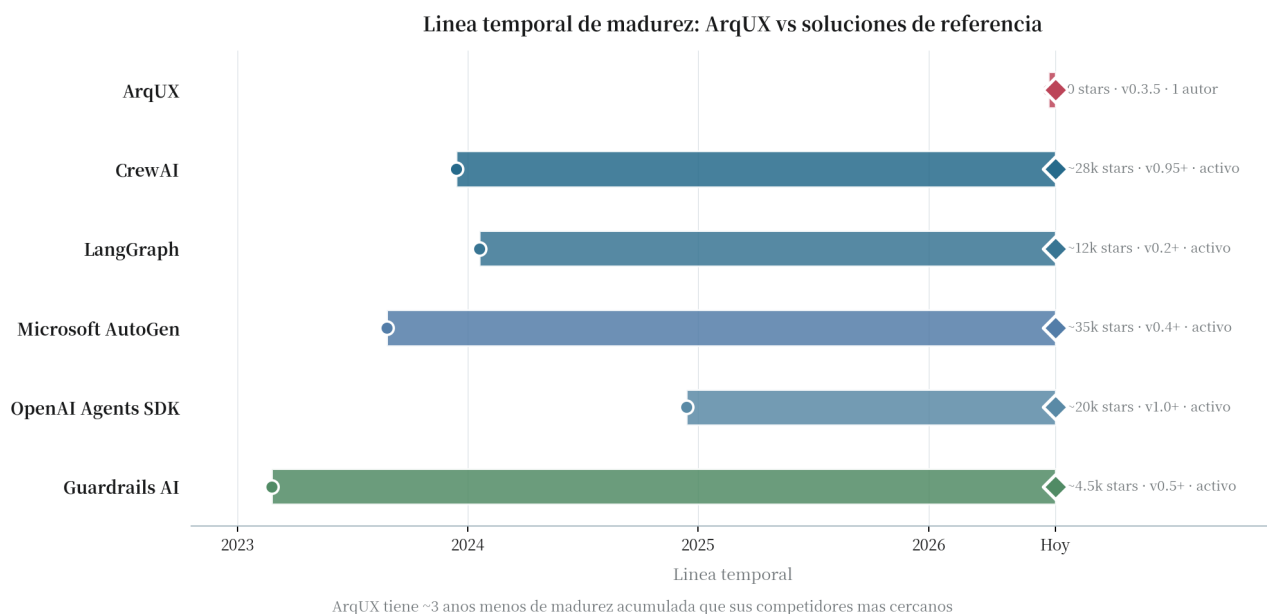


Figura 10.4 - Timeline comparativo de madurez. ArqUX tiene horas vs anos de sus competidores.

10.6 Diagrama comparativo de arquitectura

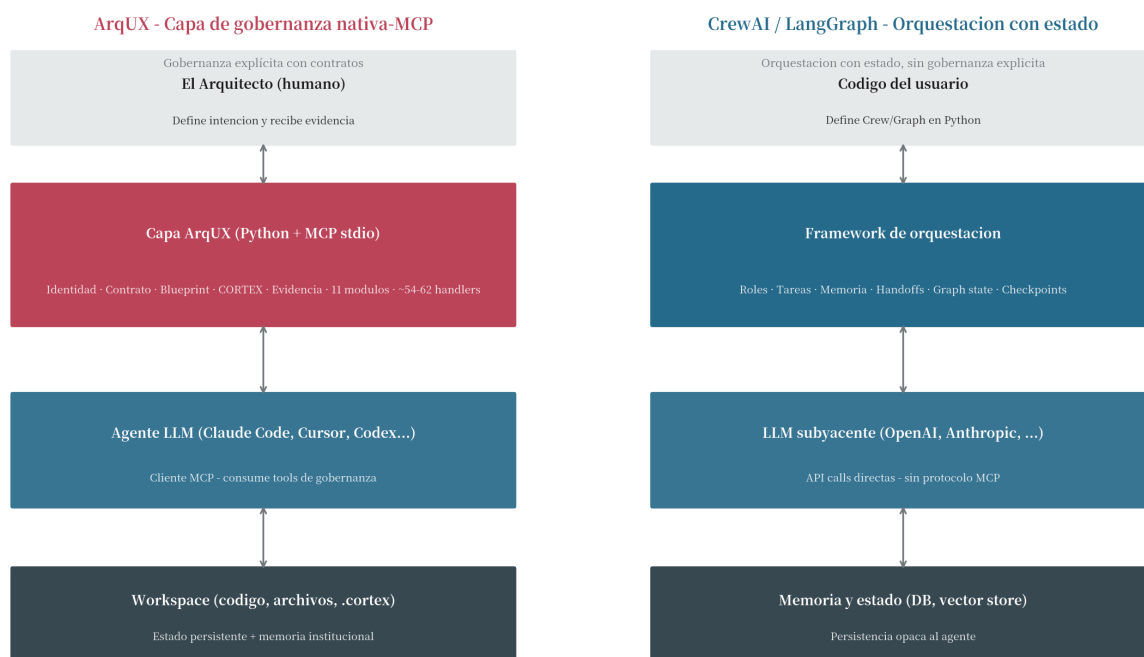


Figura 10.5 - Arquitectura ArqUX (coral, gobernanza nativa-MCP) vs CrewAI/LangGraph (azul, orquestacion con estado).

11. Hallazgos: virtudes y debilidades

11.1 Virtudes de ArqUX

ArqUX aporta cinco virtudes diferenciadoras que justifican su existencia como objeto de estudio y vigilancia, aun cuando su madurez operativa no justifique adopcion en produccion. Estas virtudes son conceptualmente significativas y no se encuentran combinadas en ninguna de las cinco soluciones de referencia evaluadas.

- **Tesis diferenciadora clara.** Es la unica solucion nativa-MCP que combina identidad + contrato + evidencia + memoria institucional en una sola capa. CrewAI tiene identidad pero no contrato; LangGraph tiene estado pero no evidencia; AutoGen tiene conversacion pero no memoria institucional; OpenAI Agents SDK tiene handoffs pero no Blueprints; Guardrails AI tiene validacion pero no identidad. La tesis de ArqUX llena un vacio real.
- **Codigo razonablemente estructurado.** El patron HandlerSpec registry es limpio y extensible. Los permisos por rol (governor/executor/auditor) estan bien modelados. Los tests existen y la cobertura auto-reportada crecio de 69 a 100 tests en CYCLE-01. El dogfooding del propio repo (carpeta .arqux/) prueba que el autor usa la herramienta, no solo la vende.
- **Stack moderno y dependencias maduras.** Python 3.10+, mcp>=1.0.0 (SDK oficial Anthropic), pydantic 2.x, click, rich. Apache-2.0. Sin dependencias exoticas ni lock-in corporativo. La unica dependencia fragil es codec-cortex, proyecto hermano del mismo autor.
- **Learning loop conceptual.** El motor de aprendizaje de tres capas (conductual -> contextual -> procedimental) aspira a convertir lecciones operativas en conocimiento institucional reutilizable. Aunque hoy no cierra el ciclo (GAP-003), la idea es original y bien formulada.
- **Plataforma-agnostica por diseno.** Funciona con cualquier cliente MCP (Claude Code, Cursor, Codex, OpenCode, Continue, Hermes). Esto lo hace inmune al riesgo de vendor lock-in con un unico proveedor de LLM.

11.2 Debilidades criticas

Las debilidades de ArqUX son estructurales y se agrupan en tres categorias: madurez operativa, salud del codigo y ecosistema. Algunas son inherentes a un proyecto de horas (adopcion, comunidad); otras son decisiones de diseno o ejecucion que pueden corregirse relativamente rapido (CI, documentacion, jerga).

- **Adopcion y comunidad nulas.** 0 estrellas, 0 forks, 0 issues, 0 menciones externas. Sin Discord, sin GitHub Discussions, sin sitio web, sin blog, sin newsletter. La busqueda web no devuelve resultados. Bus factor = 1. Esta es la debilidad mas estructural: no se resuelve con codigo, se resuelve con tiempo y construccion de comunidad.
- **Deriva documental y gaps HIGH auto-reconocidos.** Inconsistencia entre 24, 54 y 62 handlers. GAP-001 (dualidad brain.cortex) y GAP-002 (14 handlers sin documentar)

marcados como HIGH por el propio autor. CHANGELOG desincronizado (solo documenta [1.0.0] no liberado mientras la version real es 0.3.5). Renombrado reciente desde "Hermes" con referencias residuales.

- **CI insuficiente.** Tests existen pero solo corren en pipeline de release. No hay gates de test/lint/type-check en push/PR pese a tener ruff y mypy en dev deps. Esto permite que se rompa main sin detectarse.
- **Dependencia fragil.** Depende de codec-cortex (mismo autor, "specification/beta"). Si codec-cortex se abandona o cambia breaking, ArqUX se rompe. Cadena de suministro concentrada en un solo autor.
- **Sobrecarga conceptual y jerga propietaria.** Hexagon Contract, sigilos (IDN/FCS/OBJ/WRK/AXM/LIM/LNG), CORTEX/HCORTEX, BLP, CYCLE, nombres de agentes (Alfred/Jarvis/Seshat/Heimdall). La curva de aprendizaje es empinada para una ganancia funcional que podria expresarse con vocabulario estandar. Riesgo de over-engineering.
- **Quickstart superficial.** El README es manifiesto-marketing: la seccion "Quick start" son 3 lineas (pip install, arqux init, y poco mas). No hay tutorial end-to-end, no hay ejemplo de un Blueprint completo, no hay guia de migracion desde CrewAI/LangGraph. Para un framework que aspira a reemplazar conversaciones de chat, esto es un obstaculo critico.

11.3 Matriz FODA

Fortalezas	Debilidades
Tesis diferenciadora nativa-MCP unica en su categoria. Codigo estructurado con HandlerSpec registry y permisos por rol. Stack moderno: Python 3.10+, mcp>=1.0, pydantic 2.x, Apache-2.0. Dogfooding del propio repo (.arqux/). Plataforma-agnostica: cualquier cliente MCP.	Adopcion nula: 0 stars/forks/issues/menciones. Bus factor = 1, sin comunidad visible. Deriva documental: 24 vs 54 vs 62 handlers; CHANGELOG desincronizado. CI insuficiente: tests solo en release. Dependencia fragil de codec-cortex (mismo autor, beta). Jerga propietaria empinada, quickstart de 3 lineas.
Oportunidades	Amenazas
MCP se esta consolidando como estandar de facto en 2026. Demanda creciente de gobernanza de agentes en produccion. Vacio competitivo: nadie combina identidad+contrato+evidencia sobre MCP. Posibilidad de alianza con CrewAI/LangGraph como capa de gobernanza. Ecosistema Python facil de adoptar para desarrolladores AI.	Competidores con backing corporativo (Microsoft, OpenAI, LangChain Inc.) pueden añadir gobernanza nativa. Riesgo de abandono: bus factor 1 sin financiacion. Comoditizacion de guardrails en frameworks existentes reduce diferenciacion. Estandar MCP puede evolucionar y dejar a ArqUX atras si no acompaña. Adopcion de alternativas (Google A2A, etc.) puede fragmentar el mercado.

Tabla 11.1 - Matriz FODA de ArqUX v0.3.5 (captura julio 2026).

12. Recomendaciones accionables

12.1 Para equipos evaluando adoptar ArqUX

La recomendación principal es no adoptar ArqUX en producción hoy. Su madurez operativa (score 1.69/5.00, ranking #6 de 6) no justifica el riesgo. Sin embargo, esto no significa ignorarlo: ArqUX aporta ideas diferenciadoras que pueden incorporarse a la operación de equipos que ya usan CrewAI, LangGraph o AutoGen. Las siguientes recomendaciones son accionables e inmediatas.

- **No usar en producción.** Esperar al menos a v1.x con comunidad activa (>500 estrellas, >3 contribuidores externos, >1 caso de estudio público). Reevaluar en v1.1 del benchmark (Q4 2026).
- **Vigilar como referencia emergente.** Suscribirse al repo, observar evolución de gaps HIGH (GAP-001/002), evaluarlo trimestralmente. Es un buen caso de estudio para equipos que diseñan su propia capa de gobernanza interna.
- **Robar ideas, no código.** El modelo Hexagon Contract (Identity, Contract, Context, Decision, Execution, Evidence) es un checklist útil para diseñar gobernanza sobre cualquier framework. Adaptarlo a CrewAI/LangGraph sin adoptar la jerga de ArqUX.
- **Casos de uso experimentales.** Si se quiere probar, usarlo en proyectos personales o internos no críticos, con fallback a arqux call (sin MCP) para reducir dependencia. Documentar fricciones y reportarlas al autor.
- **Evitar dependencia transitiva.** Si se adopta, auditar codec-cortex separadamente; considerar pin de versión para ambos.

12.2 Para el autor de ArqUX

Las siguientes recomendaciones están priorizadas por impacto en score del benchmark v1.1. Cada una mapea a un subcriterio específico del protocolo y es medible. El autor ya ha demostrado capacidad de self-assessment honesto (archivo aprendizajes-ciclo-01.md), lo que sugiere que puede ejecutar estas recomendaciones.

- **[PRIORIDAD 1] Cerrar GAP-001 y GAP-002.** Unificar la dualidad brain.cortex y documentar los 14 handlers faltantes. Esto sube el score de Core Engineering sin trabajo arquitectónico mayor.
- **[PRIORIDAD 1] CI completo en push/PR.** Anadir workflows de test, ruff, mypy en .github/workflows/ con gates. Esto sube Operacional de 1.67 a ~3.5 en una migración de un día.
- **[PRIORIDAD 2] Quickstart real.** Reescribir la sección Quick start del README como tutorial end-to-end de 30 minutos: crear workspace, definir identidad, crear Blueprint, ejecutar, ver evidencia. Esto sube DX y baja curva de aprendizaje.
- **[PRIORIDAD 2] Reducir jerga propietaria.** Mapear sigilos a vocabulario estándar donde sea posible (IDN -> identity, FCS -> focus, AXM -> axioms, etc.). Mantener sigilos como abreviación interna pero no como superficie pública.

- **[PRIORIDAD 2] Conseguir early adopters.** Publicar en r/LocalLLaMA, Hacker News, mostrar en meetups Python/AI. Buscar 3-5 usuarios externos que aporten feedback. Sin esto, Adopcion sigue en 0.
- **[PRIORIDAD 3] Sitio de documentacion.** MkDocs Material o similar, con guías, ejemplos, API reference. Publicar en GitHub Pages. Esto sube Ecosistema significativamente.
- **[PRIORIDAD 3] Governance del proyecto.** Anadir CONTRIBUTING.md, CODE_OF_CONDUCT.md, security policy. Definir proceso de aceptacion de PRs externos. Esto reduce percepcion de riesgo empresarial.
- **[PRIORIDAD 4] Desacoplar codec-cortex.** Permitir que el formato .cortex sea opcional o sustituible. Considerar merger codec-cortex dentro de ArqUX si la separacion no aporta valor.

12.3 Para el protocolo de benchmark v1.1

El protocolo v1.0.0 es cualitativo-cuantitativo. La version v1.1 (plan: Q4 2026) debe anadir metricas cuantitativas reproducibles. Las siguientes extensiones estan priorizadas por valor agregado y costo de implementacion.

- **Benchmarks de performance reproducibles.** Medir latencia MCP, throughput de handlers, memory footprint con un harness estandar (pytest-benchmark + memory-profiler). Definir workload representativo (ej. 1000 invocaciones de handler blueprint.create).
- **Encuestas DX normalizadas.** Plantilla de encuesta a usuarios reales con escala SUS (System Usability Scale) + preguntas especificas de gobernanza. Para ArqUX, sustituir por evaluacion heuristica de DX hasta que existan usuarios.
- **Analisis estatico de codigo.** Ejecutar SAST (bandit, semgrep), medir complejidad ciclomatica (radon), debt ratio (wily). Comparar con valores de referencia del ecosistema Python.
- **Matriz de compatibilidad MCP.** Probar ArqUX contra cada cliente MCP declarado (Claude Code, Cursor, Codex, OpenCode, Continue, Hermes) y reportar compatibilidad real vs declarada.
- **Revision de pesos por dimension.** Tras 6 meses de uso del protocolo, revisar si los pesos (30/20/20/15/15) reflejan prioridades reales de adopcion. Considerar bump de Sostenibilidad a 20% si la brecha ArqUX vs grupo persiste.

13. Roadmap de evolucion del protocolo

13.1 Plan de versiones

Versio n	Fecha target	Cambios principales	Backward-co mpatible
v1.0	Julio 2026 (actual)	Protocolo inicial cualitativo-cuantitativo, 5 dimensiones, 25 subcriterios, scoring Likert 1-5 con anclajes	N/A (baseline)
v1.1	Q4 2026	Anadir benchmarks de performance reproducibles (latencia MCP, throughput, memory). Encuestas DX normalizadas. Matriz compatibilidad MCP.	Si (subcriterios nuevos opcionales)
v1.2	Q1 2027	Analisis estatico de codigo (SAST, complejidad). Comparacion de telemetria interna (si los proyectos la exponen).	Si
v2.0	Q3 2027	Posible nueva dimension (Compliance regulatorio). Revision de pesos. Scoring automatizado via dashboard vivo.	No (cambio breaking)
v2.1	Q4 2027	Integracion con telemetria real de usuarios. Bayesian updating de scores con evidencia acumulada.	Si

Tabla 13.1 - Roadmap de evolucion del protocolo de benchmark.

13.2 Dependencias y riesgos del roadmap

El roadmap depende de tres factores externos. Primero, la consolidacion de MCP como estandar: si MCP pierde traccion frente a alternativas (Google A2A, protocolos propietarios), el benchmark debe reconfigurar su categoria de referencia. Segundo, la evolucion del propio ArqUX: si el proyecto se abandona o pivota, el benchmark pierde su objeto de estudio central y debe decidir si continuar como benchmark generico de gobernanza de agentes. Tercero, la disponibilidad de telemetria publica: si los frameworks de referencia comienzan a publicar telemetria de uso, el benchmark puede anadir dimensiones cuantitativas robustas en v2.x.

13.3 Criterios de exito del protocolo

El protocolo v1.0.0 sera considerado exitoso si cumple tres criterios medibles en 12 meses: (1) al menos 3 evaluadores externos reproducen el benchmark independientemente y reportan scores dentro de ± 0.3 del score oficial; (2) el protocolo se aplica al menos a 2 frameworks adicionales a ArqUX (ej. un framework emergente MCP nuevo); (3) el autor de ArqUX reconoce utilidad del benchmark en su roadmap o decisiones de diseno. Si estos criterios no se cumplen, el protocolo requiere revision estructural en v2.0.

14. Conclusiones

El benchmark ArqUX v1.0.0 establece un piso metodológico reproducible para evaluar frameworks de gobernanza de agentes IA. Su aplicación a ArqUX arroja un resultado claro: la tesis diferenciadora del proyecto (gobernanza nativa-MCP con identidad + contrato + evidencia) es conceptualmente sólida y llena un vacío real en el ecosistema, pero su madurez empírica es insuficiente para adopción en producción. Con un score ponderado de 1.69/5.00 y ranking #6 de 6, ArqUX no califica hoy como solución industrial viable, pero sí como referencia emergente meritoria de vigilancia.

El valor del benchmark no reside únicamente en el veredicto sobre ArqUX, sino en el protocolo mismo. Las cinco dimensiones (Core Engineering, Ecosistema, Operacional, Adopción, Sostenibilidad), los 25 subcriterios operacionalizables, los anclajes Likert descriptivos y el procedimiento de reproducción de cinco pasos constituyen un asset reutilizable para evaluar cualquier framework emergente en la categoría. El roadmap de evolución (v1.1 con performance, v1.2 con análisis estático, v2.0 con compliance) asegura que el protocolo mejore con el ecosistema.

Para ArqUX específicamente, el camino hacia relevancia industrial pasa por tres decisiones críticas en los próximos 6-12 meses. Primero, cerrar los gaps HIGH auto-reconocidos (GAP-001 y GAP-002): sin esto, ni siquiera la adopción experimental es segura. Segundo, construir comunidad: 0 estrellas es un punto de partida que requiere activación intencional (publicación en HN, meetups, blog posts, colaboraciones con early adopters). Tercero, reducir la jerga propietaria y elaborar un quickstart real: la calidad del código no importa si nadie puede empezar a usarlo en 30 minutos.

Para evaluadores externos, la recomendación es usar este benchmark como piso metodológico, aplicar el protocolo a ArqUX en sus próximas versiones, y considerar su extensión a otros frameworks emergentes de gobernanza. El Anexo B contiene todas las URLs verificadas y los comandos exactos para reproducir la captura de datos. Cualquier desviación o mejora del protocolo debe documentarse en una nueva versión semver, siguiendo los criterios del Capítulo 5.8.

La próxima evaluación formal (v1.1 del benchmark) está prevista para Q4 2026. Hasta entonces, este documento constituye la referencia oficial del estado de ArqUX y del protocolo de evaluación. La invitación abierta es a la comunidad: reproducir, criticar, mejorar.

Anexo A - Glosario tecnico

Definiciones de terminos clave usados en el benchmark. La terminologia propia de ArqUX (marcada con *) es la declarada por el autor; el resto es vocabulario estandar de la industria.

Termino	Definicion
Agente gobernado*	Agente de IA que opera bajo una capa de gobernanza con identidad, contrato y evidencia, en oposicion a un agente libre sin restricciones.
API surface	Conjunto de puntos de entrada publicos que un framework expone a sus usuarios. Mide la complejidad de aprendizaje.
Blueprint lifecycle*	Ciclo de decision documentado en ArqUX con 18 secciones canonicas (Problem -> ... -> Quality) y gates de calidad con cross-verification.
Bus factor	Numero de personas clave cuyo abandono del proyecto provocaria su colapso. Bus factor 1 es el minimo riesgo.
codec-cortex*	Proyecto hermano de ArqUX, del mismo autor, que define el formato de memoria .cortex. Estado: specification/beta.
CORTEX memory*	Formato denso de memoria persistente en ArqUX para consumo del modelo. HCORTEX es la vista humana equivalente.
CI/CD	Continuous Integration / Continuous Deployment. Pipelines automatizados de test, lint, type-check y release.
Dogfooding	Practica de usar el propio producto internamente. ArqUX dogfood-ea su propio repo via la carpeta .arqux/.
HandlerSpec*	Patron registry de ArqUX: tupla (name, fn, description, input_schema) que define cada handler y se expone automaticamente como tool MCP.
Hexagon Contract*	Modelo conceptual central de ArqUX con seis fronteras: Identity, Contract, Context, Decision, Execution, Evidence.
Hot identity handoff*	Cambio de identidad de agente por invocacion natural (ej. "Hello Jarvis"). Conceptualmente similar a handoffs en OpenAI Agents SDK.
Likert 1-5	Escala ordinal de 5 puntos (1=inaceptable, 5=excelente) usada para scoring en este benchmark.
LTS	Long Term Support. Compromiso del proyecto de mantener y parchear una version por un periodo extendido.
MCP	Model Context Protocol. Estandar de Anthropic (2024) para que agentes de IA consuman tools y contexto de forma unificada.
SAST	Static Application Security Testing. Analisis estatico de codigo para detectar vulnerabilidades.
Sigilos*	Mini-DSL de memoria CORTEX: IDN (identidad), FCS (foco), OBJ (objetivos), WRK (work), AXM (axiomas), LIM (limites), LNG (lecciones), DESC (descripcion).
Stateful graph	Grafo de control con estado persistente entre nodos. Concepto central de LangGraph.

Termino	Definicion
Semver	Semantic Versioning. Esquema MAJOR.MINOR.PATCH para versionado de software.
Workflow*	Secuencia canonica de operaciones en ArqUX. Hay 10 workflows declarados (w01 workspace-init ... w10 identity-handoff).

Tabla A.1 - Glosario tecnico del benchmark (asterisco = termino propio de ArqUX).

Anexo B - Fuentes verificadas y ficha de reproducibilidad

B.1 Fuentes verificadas (captura: 2026-07-08)

Todas las URLs fueron verificadas el 8 de julio de 2026. Cualquier cambio posterior puede afectar la reproducibilidad del benchmark.

Categoría	URL	Uso
ArqUX README	https://raw.githubusercontent.com/FidelErnesto03/arqux/master/README.md	Identidad, proposito, features
ArqUX pyproject	https://raw.githubusercontent.com/FidelErnesto03/arqux/master/pyproject.toml	Stack, dependencias, classifiers
ArqUX PyPI	https://pypi.org/pypi/arqux/json	Versiones, upload_time, dependencias
ArqUX CHANGELOG	https://raw.githubusercontent.com/FidelErnesto03/arqux/master/CHANGELOG.md	Roadmap, deriva documental
ArqUX self-assessment	https://raw.githubusercontent.com/FidelErnesto03/arqux/master/aprendizajes-ciclo-01.md	Gaps HIGH, cobertura tests
ArqUX CI	https://raw.githubusercontent.com/FidelErnesto03/arqux/master/.github/workflows/release.yml	CI/CD insuficiente
ArqUX src/server.py	https://raw.githubusercontent.com/FidelErnesto03/arqux/master/src/arqux/server.py	Servidor MCP thin
ArqUX src/cli.py	https://raw.githubusercontent.com/FidelErnesto03/arqux/master/src/arqux/cli.py	CLI Click
ArqUX handlers/__init__.py	https://raw.githubusercontent.com/FidelErnesto03/arqux/master/src/arqux/handlers/__init__.py	Registry de handlers
ArqUX skills/mcp-handlers	https://raw.githubusercontent.com/FidelErnesto03/arqux/master/src/arqux/skills/mcp-handlers.skill.md	Integraciones MCP
ArqUX identities/alfred.cortex	https://raw.githubusercontent.com/FidelErnesto03/arqux/master/src/arqux/identities/alfred.cortex	Identidad Governor
codec-cortex README	https://raw.githubusercontent.com/FidelErnesto03/codec-cortex/main/README.md	Formato .cortex
codec-cortex PyPI	https://pypi.org/pypi/codec-cortex/json	Estado specification/beta
npm arqux (negativo)	https://registry.npmjs.org/arqux	Verificacion: no existe
CrewAI repo	https://github.com/crewAIInc/crewAI	Stack, estrellas, enfoque
LangGraph repo	https://github.com/langchain-ai/langgraph	Stack, estrellas, enfoque
AutoGen repo	https://github.com/microsoft/autogen	Stack, estrellas, enfoque

Categoria	URL	Uso
OpenAI Agents SDK	https://github.com/openai/openai-agents-python	Stack, estrellas, enfoque
Guardrails AI repo	https://github.com/guardrails-ai/guardrails	Stack, estrellas, enfoque

Tabla B.1 - Fuentes verificadas durante la investigacion.

B.2 Comandos para re-extraer datos

El siguiente bloque de comandos permite reproducir la captura de datos publicos usada en este benchmark:

```
# Captura de metadatos PyPI
curl -s https://pypi.org/pypi/arqux/json | jq ".info.version, .releases | keys"

# Captura de metricas GitHub via API
curl -s https://api.github.com/repos/FidelErnesto03/arqux | jq ".stargazers_count, .forks_count, .open_issues_count"

# Captura de CHANGELOG y self-assessment
curl -s https://raw.githubusercontent.com/FidelErnesto03/arqux/master/CHANGELOG.md
curl -s https://raw.githubusercontent.com/FidelErnesto03/arqux/master/aprendizajes-ciclo-01.md

# Verificacion de ausencia en npm
curl -s https://registry.npmjs.org/arqux

# Búsqueda web de menciones externas
# Usar cualquier search engine: "arqux FidelErnesto" -site:github.com -site:pypi.org
```

B.3 Plantilla de captura para futuras versiones

Para aplicar el protocolo v1.0.0 a una version futura de ArquUX (o a otro framework), usar la siguiente plantilla de captura. Documentar cada campo con timestamp y URL.

Campo	Valor capturado	URL fuente	Timestamp
Version actual	_____	_____	_____
Primer release date	_____	_____	_____
Estrellas GitHub	_____	_____	_____
Forks GitHub	_____	_____	_____
Contribuidores	_____	_____	_____
Commits ultimo mes	_____	_____	_____
Issues abiertos	_____	_____	_____
Downloads PyPI/npm mensuales	_____	_____	_____

Campo	Valor capturado	URL fuente	Timestamp
Tests (cantidad archivos)	_____	_____	_____
CI en push/PR (si/no)	_____	_____	_____
Menciones externas (count)	_____	_____	_____
Bus factor (estimado)	_____	_____	_____

Tabla B.2 - Plantilla de captura para reproduccion del benchmark.

B.4 Nota final de reproducibilidad

Este benchmark fue elaborado con evidencia 100% publica y verificable. Cualquier evaluador con acceso a internet puede reproducir la captura de datos siguiendo los comandos del bloque anterior y aplicar el protocolo del Capitulo 5 para obtener scores comparables. Las desviaciones respecto a los scores reportados aqui deben documentarse y discutirse; diferencias mayores a 0.5 puntos en cualquier dimension sugieren necesidad de clarificar anclajes en v1.1 del protocolo.

Para feedback, propuestas de mejora del protocolo, o reportes de reproduccion, abrir issue en el repositorio del benchmark (pendiente de publicacion). La version 1.0.0 es el baseline; toda mejora posterior debe seguir semver y documentarse en un CHANGELOG propio del protocolo.